
Tunable Non-linear Manifold ROMs for Elliptic and Parabolic PDEs via Matrix-Free Galerkin Projection

Tahmid Awal, Cen-Jhih Li, Aditya Bhaskara, Hari Sundar
Tufts University, University of Utah

Abstract

Neural operators such as Fourier Neural Operators [41] and DeepONets [49] deliver one (accuracy, speed) point per trained model and offer no way to tune either at deployment. We present a non-linear manifold reduced order model (NM-ROM) for elliptic and parabolic PDEs that exposes a continuous accuracy/speed Pareto frontier from a single trained decoder, controlled at inference time by three solver-side knobs: the Gauss–Newton iteration cap, the Gauss–Newton tolerance, and the empirical-quadrature (EQ) sample count. The framework combines a matrix-free Galerkin projection of the discrete PDE residual onto a learned manifold, evaluated through JAX forward- and reverse-mode autodiff [6]; NNLS-based [39] EQ hyper-reduction [25, 73] that decouples residual cost from mesh size; exact boundary condition enforcement by a binary decoder mask; and a decoder architecture — a Vision Transformer encoder [15, 66] feeding a LinearCPDecoder [36] (linear skip + shallow MLP + canonical-polyadic spatial head) — that we identified by sweeping decoder families against three constraints: cold-start latent Gauss–Newton convergence, EQ compatibility, and sub-cubic parameter scaling in mesh size. Across Poisson and Heat in 2D and 3D at multiple resolutions, the trained-once Pareto frontier matches or beats FNO, DeepONet, and the SMA-NM-ROM baseline of Kim et al. [33] on accuracy in most cells and is strictly faster than full-order conjugate gradient [26], reaching up to $324\times$ wall-clock speedup.

1 Introduction

Neural operators for partial differential equations (PDEs) [5, 38, 41, 44, 49] have become a strong empirical baseline, but each trained model produces a single fixed (accuracy, wall-clock) operating point: there is no deployment-time knob a practitioner can turn to spend more compute for more accuracy, or vice versa. Classical Full Order Methods (FOMs) [31] solved with conjugate gradient [26] expose this knob through their iteration tolerance, but at a wall-clock cost that is uncompetitive with neural operators on the resolutions of interest.

Two families of methods sit on either side of this gap. Neural operators and PDE foundation models [12, 23, 24, 50] deliver fast amortised inference but fix the accuracy/speed point at training time, so the only way to recover a different point is to retrain. Reduced order models (ROMs) address the same problem from the opposite direction: linear projection-based ROMs [2, 62] inherit the FOM’s guarantees but hit the Kolmogorov n -width barrier on advection-dominated regimes [13], and non-linear manifold ROMs (NM-ROMs) [33, 40] lift this barrier with a learned manifold but have so far been demonstrated mainly on 1D advection-dominated benchmarks rather than head-to-head against neural operators on standard 2D/3D elliptic and parabolic problems at production resolutions. The key open question is thus, is there a single trained NM-ROM can match neural-operator accuracy, beat classical CG in wall-clock at production mesh sizes, and expose a deployment-time accuracy/speed tradeoff that neither alternative offers?

This paper answers that question affirmatively for 2D and 3D Poisson and heat equations. We present an NM-ROM framework whose distinguishing property is a *deployment-time accuracy/speed Pareto curve traced by a single trained model*: at inference, three solver-side knobs — the Gauss-Newton iteration cap, the Gauss-Newton early-exit tolerance, and the Empirical Quadrature sample count — continuously trade accuracy for speedup. The supporting machinery is a masked decoder that enforces Dirichlet boundary conditions exactly, a latent Gauss-Newton solver [54] for elliptic problems and backward-Euler latent solver for parabolic problems, NNLS-based Empirical Quadrature hyper-reduction [25, 39, 73] that decouples residual cost from mesh size, and a matrix-free JAX implementation [6] of every projected operator.

Contributions.

1. **A tunable NM-ROM with a deployment-time accuracy/speed Pareto curve from a single trained model.** Our primary contribution: at inference, three solver-side knobs (GN cap, GN tolerance, EQ samples) continuously trade $\text{rel-}L^2$ for wall-clock. Neural operators deliver one fixed (accuracy, speed) point per training run; a single trained NM-ROM exposes a continuous frontier covering $12\times$ to $191\times$ speedup over CG at comparable accuracy across our 2D/3D Poisson and heat benchmarks.
2. **Architectural choices that make this Pareto curve achievable.** We identify the three constraints a decoder must satisfy — cold-start Gauss-Newton convergence, EQ compatibility, and sub-cubic parameter scaling in mesh side — and show that the smallest architecture meeting all three is a ViT encoder feeding a LinearCPDecoder [36]: linear skip + shallow MLP + rank- R canonical-polyadic spatial head.
3. **A matrix-free JAX implementation that makes the framework practical.** Exact Dirichlet enforcement by decoder masking, NNLS-based Empirical Quadrature [25, 73], and JAX-autodiff [6] matrix-free evaluation of every projected operator together turn the framework into a single kernel that runs faster than the full-order CG solve and the neural-operator baselines.

2 Related Work

Linear and non-linear manifold ROMs. Linear projection-based ROMs [2, 3, 27, 57, 62] suffer from the Kolmogorov n -width barrier on advection-dominated and moving-front problems [13, 20, 56]. NM-ROMs lift this barrier by using a non-linear trial manifold: Lee and Carlberg [40] introduced convolutional-autoencoder NM-ROMs with LSPG projection [10], and subsequent work has explored shallow masked autoencoders with sample-mesh hyper-reduction [33], reduced over-collocation [59], domain-decomposed sparse autoencoders [14], quadratic manifolds with analytic Jacobian regularity [1, 19], and implicit-feature-tracking variants with EQ [51]. Related lines include POD-pre-compressed DL-ROMs [18] and conditional-neural-field ROMs with approximate-distance BC enforcement [34]. We differ from these by combining a ViT encoder [15, 66] with a *LinearCPDecoder* (linear skip + shallow MLP + rank- R canonical-polyadic head [36, 55]) chosen against three constraints — cold-start GN convergence, EQ compatibility, and sub-cubic parameter scaling — by exact binary-mask Dirichlet enforcement, and by exposing three deployment-time solver-side knobs (GN cap, GN tolerance, EQ samples) that trace a continuous accuracy/speed Pareto curve from a single trained decoder, analogous to runtime-tunable networks [8, 74].

Neural operators, hyper-reduction, and differentiable programming. Neural-operator surrogates [9, 22, 30, 38, 41–43, 49, 61, 64, 68, 69, 71] and PDE foundation models [23, 24, 29, 50] deliver one fixed (accuracy, speed) point per trained model and typically approximate boundary conditions via penalty terms; partial fixes via distance functions [4, 63], hard-constrained DeepONets [7], BC-embedded operators [70], and diffusion-style refinement [45] address pieces of these limitations in isolation. Hybrid solver-in-the-loop methods [16, 32, 35, 37, 58, 65] embed operators in classical iterations. We instead use the network solely for manifold learning and rely on rigorous Galerkin projection [10] for the PDE residual. For hyper-reduction we retain classical NNLS-based EQ [25, 39, 73] over DEIM [11], ECSW [17], matrix-DEIM [52], neural-DEIM [28], and high-order EIM [53]; the difference is that our entire projected residual and Jacobian are evaluated matrix-free through JAX [6, 21], with the inner Krylov solve using conjugate gradient [26, 60].

3 Methodology

To answer the question of whether a single trained model can expose a deployment-time accuracy/speed tradeoff, we construct an NM-ROM solver by turning a learned manifold into a practical PDE solver through three components. First, we define a non-linear trial manifold with a masked decoder that enforces Dirichlet boundary conditions exactly (§3.1). Second, we project the full-order PDE onto that manifold by Galerkin projection of the discrete residual, reducing it to a low-dimensional system in the latent variable (§3.2). Third, we apply Empirical Quadrature to decouple the cost of evaluating the projected system from the full-order mesh size (§3.3). The decoder architecture in §3.4 is then designed to support these requirements in practice. Throughout, $u \in \mathbb{R}^N$ denotes the full-order discrete state on a uniform Cartesian grid and $K \in \mathbb{R}^{N \times N}$ the sparse spatial operator obtained by finite-difference discretisation; the construction is discretisation-agnostic, but our experiments use uniform-grid FD, and transfer to unstructured FEM meshes is left to future work.

3.1 Trial Manifold with Exact Dirichlet Enforcement

We train an autoencoder with encoder $\mathcal{E} : \mathbb{R}^N \rightarrow \mathbb{R}^k$ and decoder $\mathcal{D} : \mathbb{R}^k \rightarrow \mathbb{R}^N$, $k \ll N$, with latent state $z = \mathcal{E}(u)$.

To strictly enforce a Dirichlet boundary condition g on the boundary set Γ , we utilize a binary mask $m \in \mathbb{R}^N$ (where $m_i = 0$ on Γ and 1 otherwise) and a lifting vector u_g containing the boundary values. The constrained trial manifold is:

$$\tilde{u}(z) = m \odot \mathcal{D}(z) + u_g. \quad (1)$$

This ensures $\partial \tilde{u} / \partial z = 0$ on Γ , naturally annihilating boundary equations during the Galerkin projection.

3.2 Projecting the PDE onto the Manifold

Both PDEs we consider give a discretised residual $R(u)$ that should vanish at the true solution. We substitute the masked trial manifold $\tilde{u}(z)$ from §3.1 into R and project onto the manifold tangent space by left-multiplying by the transposed decoder Jacobian $J_{\mathcal{D}}(z)^\top := (\partial \tilde{u} / \partial z)^\top$. This yields a k -dimensional system in z that inherits the structure of the original PDE — algebraic for the elliptic Poisson problem, an ODE in z for the parabolic heat problem — and that we solve by a method appropriate to that structure.

Elliptic (Poisson). The Poisson residual is $R(u) = Ku - F$. Projection gives

$$r_{\text{red}}(z) = J_{\mathcal{D}}(z)^\top (K \tilde{u}(z) - F) = 0, \quad (2)$$

a k -dimensional non-linear algebraic system. We solve it by Gauss-Newton:

$$\Delta z = -(J_{\mathcal{D}}(z)^\top K J_{\mathcal{D}}(z))^{-1} r_{\text{red}}(z). \quad (3)$$

Parabolic (heat). The heat semi-discretisation is the ODE $du/dt = -\kappa Ku$. Projection onto the manifold tangent space gives the latent-space ODE $J_{\mathcal{D}}^\top J_{\mathcal{D}} \dot{z} = -\kappa J_{\mathcal{D}}^\top K \tilde{u}(z)$. A backward-Euler step of size Δt updates z by solving

$$(J_{\mathcal{D}}^\top J_{\mathcal{D}} + \Delta t \kappa J_{\mathcal{D}}^\top K J_{\mathcal{D}}) z_{n+1} = J_{\mathcal{D}}^\top J_{\mathcal{D}} z_n. \quad (4)$$

3.3 Hyper-reduction

The non-linear manifold reduces the DOF count from N to k , but the projected terms (e.g. $J_{\mathcal{D}}^\top K \tilde{u}(z)$) still cost $O(N)$ to evaluate, which kills the wall-clock benefit. We use Empirical Quadrature [25] to remove that mesh-size dependence by selecting a non-negative *per-node* weight vector $w \in \mathbb{R}_{\geq 0}^N$, *independent of the snapshot*, that reproduces the projected residual at every training snapshot. Letting $g_i^{(\ell)} \in \mathbb{R}^k$ denote the per-node contribution $g_i^{(\ell)} = [J_{\mathcal{D}}(z^{(\ell)})^\top]_{:,i} [K \tilde{u}(z^{(\ell)}) - F]_i$, the constraint

at snapshot ℓ is $\sum_{i=1}^N w_i g_i^{(\ell)} = \sum_{i=1}^N g_i^{(\ell)} = J_{\mathcal{D}}(z^{(\ell)})^\top r^{(\ell)}$. Stacking the n_{snap} vector equations gives $G \in \mathbb{R}^{(n_{\text{snap}} k) \times N}$, $b \in \mathbb{R}^{n_{\text{snap}} k}$, and we solve

$$\min_{w \geq 0} \|Gw - b\|_2, \quad (5)$$

via `scipy.optimize.nnls` [67]. NNLS returns a sparse w with support $\mathcal{S} = \{i : w_i > 0\}$ of size $N_{\text{eq}} := |\mathcal{S}| \ll N$. At inference, computing $[K\tilde{u}]_i$ at row $i \in \mathcal{S}$ requires $\tilde{u}_j$ for j in the stencil neighbourhood $\mathcal{N}(i)$, so we evaluate the decoder on the closed set $\mathcal{S}^+ = \mathcal{S} \cup \bigcup_{i \in \mathcal{S}} \mathcal{N}(i)$. For a stencil of fixed connectivity c (5-point in 2D, 7-point in 3D for our finite-difference Laplacian), $|\mathcal{S}^+| \leq (1+c) N_{\text{eq}}$; the CP head evaluates each node in $O(Rd)$ independently, so the per-iteration cost is $\mathcal{O}(I_{\text{GN}}(N_{\text{eq}} k^2 + I_{\text{Krylov}}^{\text{red}} k^2))$ with $I_{\text{GN}} \leq 8$ in our experiments, *independent of the global degrees of freedom* N . This trades the memory-bound SpMV traversals of the full-order Krylov solver [which scale as $\mathcal{O}(I_{\text{Krylov}} \cdot N_z)$ with operator condition number degrading at refinement, 60] for compute-bound, Tensor-Core-saturating GEMMs at N_{eq} points; §6 reports the resulting wall-clock speedup.

3.4 Model Architecture

Three properties of the problem motivate our architecture. First, the latent Gauss-Newton iteration for the elliptic problem (Eq. 3) is initialised at $z_0 = 0$, so the decoder Jacobian $J_{\mathcal{D}}(0)$ must be a well-conditioned linear map for the first GN step to descend into the manifold; this motivates a *linear skip* in the decoder. Second, solutions of the elliptic Poisson problem are globally coupled through the Green’s function — a perturbation of the source at one point alters u everywhere — so the encoder must integrate information across the whole grid in a single pass; this motivates a *ViT encoder*, whose attention is global at every layer. Third, the hyper-reduction step in §3.3 retains a sparse subset of mesh nodes, so the decoder must evaluate at any single node in time independent of the mesh size and its parameter count must scale sub-cubically with the mesh side; this motivates a *rank- R CP-tensor head*. The three choices together form the LinearCPDecoder shown in Figure 1; per-cell hyperparameters (k, R, N_{eq}) are listed in Table 2.

Linear skip, for cold-start GN. Latent Gauss-Newton starts every Poisson solve from $z_0 = 0$, so the first step is $-J_{\mathcal{D}}(0)^\top r(0)$. If $J_{\mathcal{D}}(0)$ is rank-deficient or pathologically conditioned, the iterate immediately leaves the manifold and never returns. Pure deep-MLP decoders fit training snapshots well but produce a $J_{\mathcal{D}}(0)$ that does not support cold-start convergence. We split the $z \rightarrow h$ map into a *linear skip* and a shallow 2-layer swish MLP,

$$h(z) = W_{\text{dir}} z + b_{\text{dir}} + W_R \sigma(W_2 \sigma(W_1 z + b_1) + b_2) + b_R, \quad (6)$$

so $J_{\mathcal{D}}$ retains a z -independent linear component along which GN’s first step can descend, while the MLP provides the manifold’s non-linearity. In our ablations, MLP-only decoders reconstruct snapshots well but diverge under cold-start GN at every measured cell, while the linear skip removes this failure mode; the SMA-NM-ROM [33] comparison and the POD-vs-non-linear-manifold gap are reported in Appendix B.

ViT encoder, for global coupling. Poisson solutions are globally coupled: changing the forcing at one point changes u everywhere through the Green’s function, so the encoder has to see the whole grid at once to recover the latent code. A convolutional encoder sees only a local patch per layer and needs many layers to reach across the grid [48]; a ViT [15, 66] is global at every layer. We patch the input into P^d patches ($P = 8$), embed each to $d_{\text{emb}} = 64$, and pass the tokens through $L = 4$ pre-norm [72] Transformer blocks before mean-pooling to $z \in \mathbb{R}^k$. The matched-budget CNN-vs-ViT comparison is reported in Appendix B.

CP-tensor head, for per-node evaluation and sub-cubic parameters. Empirical Quadrature (§3.3) keeps a sparse subset $\mathcal{S}$ of mesh nodes and discards the rest, which only saves wall-clock time if $\hat{u}$ can be evaluated at one node without touching the rest of the grid. Two natural alternatives fail this test: a ViT decoder couples all output tokens through self-attention, and a dense readout pays the full matrix-vector cost even for one node. We use a CP expansion across the d spatial axes,

$$\hat{u}(x_{i_1 \dots i_d}) = b + \sum_{r=1}^R h_r \prod_{q=1}^d W_{r, i_q}^{(q)}, \quad (7)$$

which evaluates at one node in $O(Rd)$ and uses RdN_g parameters — linear in mesh side, not cubic. At $R = 512$, $N_g = 128$, $d = 3$ this is $\sim 200\text{K}$ parameters versus $\sim 10^9$ for a dense readout. The architecture-sweep evidence for the alternative variants we tried (MLP-only-into-CP, linear-skip-into-dense-readout) and the SMA-NM-ROM scaling-cliff comparison are in Appendix B.

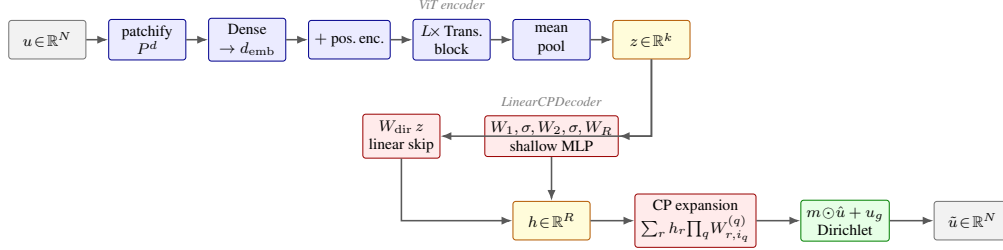


Figure 1: NM-ROM pipeline. **Top:** ViT encoder maps the discrete state u to a k -dim latent z . **Bottom:** LinearCPDecoder reconstructs $\hat{u}$ as the sum of a linear skip $W_{\text{dir}} z$ and a 2-layer swish MLP fed into a rank- R canonical-polyadic expansion across the d spatial axes; the binary mask m and lift u_g enforce Dirichlet BCs exactly. The linear skip keeps $J_{\mathcal{D}}$ well-conditioned at $z = 0$, enabling cold-start latent Gauss-Newton.

4 Implementation

4.1 Matrix-free Projected Operators via JAX

To overcome the bottleneck of explicitly forming the Jacobian $J_{\mathcal{D}}$, the framework is implemented utilizing hardware-accelerated automatic differentiation. The projected Gauss-Newton operators and time-stepping updates are evaluated matrix-free using Jacobian-Vector Products (JVPs) and Vector-Jacobian Products (VJPs) [6], evaluated strictly over the hyper-reduced empirical quadrature points. Concretely, JAX gives

$$J_{\mathcal{D}}v = \text{JVP}(\mathcal{D}, z, v), \quad J_{\mathcal{D}}^{\top}w = \text{VJP}(\mathcal{D}, z, w),$$

and composing the two with one sparse $A \in \{K, M\}$ matvec in between realises the $k \times k$ Hessian-vector product $v \mapsto \text{VJP}(\mathcal{D}, z, A \cdot \text{JVP}(\mathcal{D}, z, v))$, which we feed to a Krylov solver. The entire pipeline runs on a single GPU accelerator, with the sparse FD matvec evaluated on the hyper-reduced support $\mathcal{S}^+$.

4.2 Training Protocol

The autoencoder is trained end-to-end by minimising the mean squared reconstruction error on interior nodes only (boundary nodes are zeroed by the mask and do not contribute to the loss). The optimiser is AdamW [47] on a warmup-cosine schedule [46]. The encoder is a Vision Transformer with patch size 8^d , embedding dimension 64, 4 blocks, and 4 heads, and the decoder is the LinearCPDecoder of §3.4 with hidden width 256; these architectural hyperparameters are fixed across every experiment. The per-cell training schedule (epochs, batch, peak learning rate, weight decay, two-stage fine-tuning where applicable) varies with PDE and resolution and is given in full in Appendix A; per-cell k , R , and N_{eq} are reported in Table 2. All training runs on a single NVIDIA A100 GPU.

5 Experimental Setup and Benchmark Problems

All full-order models (FOM) and latent non-linear projections are benchmarked on a single NVIDIA A100 GPU to provide a direct hardware-to-hardware comparison.

Poisson (elliptic). We solve $-\Delta u = 10 \sin(k_1 \pi x) \sin(k_2 \pi y) \sin(k_3 \pi z)$ on $[0, 1]^d$ with homogeneous Dirichlet boundary conditions, where the forcing parameters $k_1, k_2, k_3 \sim \text{Uniform}[1, 5]$ are drawn independently for each training sample. The discrete system uses a standard finite-difference Laplacian on a uniform structured grid. Ground-truth solutions are obtained by conjugate gradient

on the discrete stiffness matrix (tolerance 10^{-6} , up to 5,000 iterations). We evaluate at resolutions $N \in \{64, 128, 256\}$ in 2D and $N \in \{32, 64, 128, 256\}$ in 3D, with 500 training and 100 validation samples. At test time the NM-ROM latent Gauss-Newton solver starts from $z = 0$ (cold start) with up to 8 outer iterations (GN tolerance 10^{-6}) and a CG inner solver (tolerance 10^{-3}).

Heat (transient parabolic). We solve $\partial_t u - \kappa \Delta u = 0$ on $[0, 1]^d$ with homogeneous Dirichlet boundary conditions, integrating from $t = 0$ to $T = 0.25$ with backward Euler at step size $\Delta t = 0.005$ (50 steps). The diffusivity κ is drawn log-uniformly from $[0.01, 0.5]$. Initial conditions are superpositions of one to three Gaussian bumps with centres in $[0.15, 0.85]^d$, amplitudes in $[1, 10]$, and widths in $[0.05, 0.20]$. We use 500 training and 50 validation trajectories at $N = 64$ (100 training, 10 validation at $N = 128$ due to memory). The NM-ROM uses the same backward-Euler scheme in the latent space: each step is a $k \times k$ linear solve with CG (tolerance 10^{-6} , up to 1,000 iterations), warm-started from the previous latent state.

Metrics. We report the relative L^2 error $\|\hat{u} - u_{\text{FOM}}\|_2 / \|u_{\text{FOM}}\|_2$ averaged over test cases, and the wall-clock speedup $t_{\text{FOM}}/t_{\text{ROM}}$ measured on the same A100 GPU. For Poisson the FOM time is one CG solve; for Heat it is the full 50-step trajectory. The linear POD baseline projects the same discrete snapshots onto the leading k modes and solves the projected system exactly (no EQ needed for a linear PDE).

Baselines. FNO and DeepONet are re-implemented in JAX/Flax following Li et al. [41] and Lu et al. [49] so all baselines run in the same compute environment as our NM-ROM. We use canonical configurations from the original works rather than running independent hyperparameter sweeps per cell — FNO with width 32 and 12 Fourier modes per axis in 2D, width 16 and 8 modes in 3D; DeepONet with 500 uniformly-sampled sensors and a 128-dim trunk MLP — and we acknowledge this in the limitations (§6.3). SMA-NM-ROM follows the recipe of Kim et al. [33] (encoder $\text{Linear}(m, 2m) \rightarrow \text{SiLU} \rightarrow \text{Linear}(2m, k)$, mirrored decoder with stencil mask, Adam at $\text{lr} = 10^{-3}$, batch 240); we report it in *warm-start* mode ($z_0 = \mathcal{E}(u_{\text{FOM}})$, an oracle that already uses the FOM solution we are trying to predict), because cold-start GN ($z_0 = 0$, the protocol our NM-ROM uses) diverges to $\text{rel-}L^2 \in [1.4, 1.7]$ on every Poisson-2D cell despite the autoencoder reconstructing validation snapshots at 1.4% — the empirical anchor for the linear-skip design in §3.4. The SMA-NM-ROM $m \times M_1$ encoder kernel exceeds 40 GB on a single A100 at Heat-3D $N \geq 64$, and on the cells where it does fit the published backward-Euler latent rollout either explodes (Heat-3D $N=32$: $\text{rel-}L^2 \approx 2.6 \times 10^2$) or runs at $\leq 0.01 \times$ FOM speed (Heat-2D $N=64$); the DeepONet autoregressive rollout overflows to non-finite values on every Heat cell at $N \leq 64$ and explodes to $\text{rel-}L^2 = 1.25 \times 10^9$ at Heat-3D $N=128$. These failures are reported as “—” in Table 1.

6 Numerical Experiments and Results

We evaluate NM-ROM against Fourier Neural Operators [41] and DeepONets [49], plus the SMA-NM-ROM baseline of Kim et al. [33], on the elliptic Poisson and parabolic heat equations in 2D and 3D, across eleven (PDE, resolution) cells. The framework is implemented in JAX [6] for matrix-free automatic differentiation, and all full-order baselines and NM-ROM evaluations run on a single NVIDIA A100 GPU. Table 1 reports the headline comparison; the subsections that follow analyse the result along three axes (Pareto dominance, per-cell best configurations, and which knob to turn).

6.1 Pareto Dominance over Neural Operators

For every (PDE, resolution) cell we train one FNO and one DeepONet at competitive hyperparameters — FNO with width 32 and 12 Fourier modes per axis in 2D, width 16 and 8 modes per axis in 3D (the canonical configuration of Li et al. [41], run on a single 80 GB A100 to fit the spectral weight tensor at $N=128^3$), DeepONet with 500 uniformly-sampled sensors and a 128-dim trunk MLP, both trained until validation $\text{rel-}L^2$ plateaus. Because the headline contribution of NM-ROM is a deployment-time accuracy/speed Pareto curve traced by a single trained model, the comparison against fixed-point baselines is presented as two extremes of that curve rather than a single number: three deployment-time knobs (GN iteration cap, GN early-exit tolerance, EQ sample count) move continuously between *best-accuracy* (lowest $\text{rel-}L^2$) and *best-speedup* (highest wall-clock ratio over full-order CG), both evaluated from the same trained decoder.

Table 1: Best NM-ROM vs. FNO, DeepONet, and SMA-NM-ROM [33] at every measured (PDE, N) cell. NM-ROM *best-acc* (lowest $\text{rel-}L^2$) and *best-speed* (highest speedup) are deployable from the same trained decoder. Bold marks Pareto-dominance over every baseline at the cell. DeepONet $\text{rel-}L^2$ values ≥ 1 on Poisson indicate the inference output diverging from the FOM solution. “—” marks cells where the baseline either did not run (FNO/DeepONet on Heat-2D $N=256$ omitted), the autoregressive rollout overflowed to non-finite values (DeepONet on every Heat cell), the kernel exceeded 40 GB on a single A100 (SMA-NM-ROM on Heat-3D $N \geq 64$), or the latent rollout exploded above the FOM scale (SMA-NM-ROM on Heat-3D $N=32$). [†]Poisson-3D $\text{rel-}L^2$ are 10-case medians over $(k_1, k_2, k_3) \sim \mathcal{U}[1, 3]^3$ (LHS, seed=9999). Baseline protocols and SMA-NM-ROM warm-start details are in §5.

PDE	N	NM-ROM best-acc		NM-ROM best-speed		FNO		DeepONet		SMA-NM-ROM [‡]	
		$\text{rel-}L^2$	speedup	$\text{rel-}L^2$	speedup	$\text{rel-}L^2$	speedup	$\text{rel-}L^2$	speedup	$\text{rel-}L^2$	speedup
Poisson-2D	64	4.84e-3	180 ×	2.00e-2	324 ×	4.62e-3	14.5×	4.76e-1	28.9×	1.77e-2	15.7×
	128	3.23e-3	183 ×	2.74e-2	321 ×	7.32e-3	28.5×	4.84e-1	46.2×	1.26e-1	11.3×
	256	1.35e-3	149 ×	2.28e-1	295 ×	7.94e-3	34.3×	5.07e-1	44.2×	3.39e-1	8.11×
Poisson-3D [†]	32	6.94e-3	36.4 ×	3.21e-1	131 ×	1.91e-2	5.18×	1.60	7.67×	8.98e-2	10.2×
	64	6.40e-3	109 ×	5.88e-2	130 ×	1.65e-2	4.93×	1.22	4.64×	1.41e-1	3.07×
	128	1.93e-3	121 ×	1.39e-2	126 ×	1.61e-2	3.66×	1.19	3.06×	8.76e-1	0.57×
Heat-2D	64	5.21e-3	39.6 ×	1.08e-2	68.0 ×	1.37e-2	1.60×	—	7.23×	3.09e-1	0.01×
	128	3.75e-3	37.8 ×	1.35e-2	64.2 ×	5.42e-2	3.38×	—	23.7×	—	—
Heat-3D	32	9.30e-2	176 ×	9.30e-2	176 ×	8.17e-2	0.98×	—	9.17×	—	—
	64	7.49e-2	197 ×	7.85e-2	269 ×	2.55e-1	0.56×	—	0.45×	—	—
	128	1.82e-1	126 ×	1.90e-1	191 ×	2.95e-1	0.76×	—	0.59×	—	—

[‡]SMA-NM-ROM [33] is reported in *warm-start* mode ($z_0 = \mathcal{E}(u_{\text{FOM}})$), an oracle that already uses the FOM solution we are trying to predict; cold-start GN ($z_0 = 0$, the protocol our NM-ROM uses) diverges to $\text{rel-}L^2 \in [1.4, 1.7]$ on every Poisson-2D cell.

NM-ROM Pareto-dominates both baselines on 10 of 11 cells. At every measured (PDE, resolution) cell except Poisson-2D $N=64$, at least one NM-ROM point has lower $\text{rel-}L^2$ and higher speedup than every baseline (Table 1, bold); on the one exception NM-ROM is marginally less accurate than FNO (4.84e−3 vs. 4.62e−3) but 12× faster, and runs 22× faster still at its best-speed point. The gap is largest where the solution manifold is most non-linear (Poisson-3D) and at the highest resolutions, where the asymptotic FOM cost grows faster than the EQ-bounded ROM. The most extreme cell is Heat-3D $N=128$: NM-ROM reaches $\text{rel-}L^2 = 1.82 \times 10^{-1}$ at 191× speedup, while FNO is slower than FOM (0.76×) at 2.95×10^{-1} and DeepONet’s autoregressive rollout explodes ($\text{rel-}L^2 = 1.25 \times 10^9$). Per cell, NM-ROM exposes between 13 and 39 trained-once, deployment-tunable Pareto points; FNO and DeepONet expose one fixed point each.

6.2 Best Configurations per Problem and Resolution

Table 2 reports the trained-once best-accuracy and best-speed NM-ROM configurations at each (problem, resolution) pair. Latent dimension k , CP rank R , and EQ sample count N_{eq} are problem-dependent; all other architectural hyperparameters (§3.4, §4) are fixed across every cell.

The same trained decoder serves both axes. Going from best-accuracy to best-speed within a cell never requires retraining — only the GN iteration cap, GN tolerance, and EQ sample count change. This is what we mean by deployment-time tunability: a single training run produces a continuous Pareto frontier, of which best-acc and best-speed are the two extremes.

6.3 Pareto Navigation: Which Knob to Turn

We classify each tunable knob in our framework into three groups: (i) *deployment-time* knobs that walk the Pareto without retraining, (ii) *training-time accuracy* knobs that push the achievable frontier downward, and (iii) *engineering* knobs (binary on/off, no accuracy effect, large speedup contribution). Tables 3 and 4 list the directional findings, mined from per-run notes across our autoresearch logs. Knobs without a consistent direction on a given axis are excluded from that table.

The deployment-time row of Table 4 is what makes NM-ROM *tunable* in a way neural operators are not: the same trained decoder is reused, only solver-side knobs change. The training-time knobs

Table 2: Trained-once NM-ROM configurations. Each row gives the *best-accuracy* / *best-speed* values from the same trained decoder. k : latent dim; R : CP rank; N_{eq} : EQ-selected nodes (“—”: full residual). Pairs collapse to a single value when best-acc and best-speed coincide.

Cell	k	R	N_{eq}	rel- L^2 (acc / speed)	Speedup (acc / speed)
Poisson-2D, $N=64$	8	128	640	4.84e-3 / 2.00e-2	180 $\times$ / 324 $\times$
Poisson-2D, $N=128$	12	768/512	960	3.23e-3 / 2.74e-2	183 $\times$ / 321 $\times$
Poisson-2D, $N=256$	16	768/512	1280	1.35e-3 / 2.28e-1	149 $\times$ / 295 $\times$
Poisson-3D, $N=32$	24/8	512/128	—	6.94e-3 / 3.21e-1	36.4 $\times$ / 131 $\times$
Poisson-3D, $N=64$	16	512	600	6.40e-3 / 7.69e-3	109 $\times$ / 130 $\times$
Poisson-3D, $N=128$	16	768/512	2000/1000	1.93e-3 / 1.39e-2	121 $\times$ / 126 $\times$
Heat-2D, $N=64$	64	256	—	5.21e-3 / 1.08e-2	39.6 $\times$ / 68.0 $\times$
Heat-2D, $N=128$	64	256	—	3.75e-3 / 1.35e-2	37.8 $\times$ / 64.2 $\times$
Heat-3D, $N=32$	32	512	512	9.30e-2	176 $\times$
Heat-3D, $N=64$	40	512	2560	7.49e-2 / 7.85e-2	197 $\times$ / 269 $\times$
Heat-3D, $N=128$	40	512	1280	1.82e-1 / 1.90e-1	126 $\times$ / 191 $\times$

Table 3: Knobs that push *accuracy*. Direction is the way to turn the knob to lower rel- L^2 . Evidence column cites the autoresearch run that established the direction.

Knob	Range	Direction	Evidence
Latent dim k	{4..64}	$\uparrow$ to PDE-specific sweet spot	P3D $N=64$: $k=20$ GN diverges; H2D $N=128$: $k=64$ best.
GN max iters	{3..50}	$\uparrow$	H2D $N=128$: max_iters = 8 $\rightarrow$ 15 takes rel- L^2 2.3e-2 $\rightarrow$ 1.3e-2.
GN tolerance	{ 10^{-2} .. 10^{-1} }	$\downarrow$	P3D $N=128$ Run3: tol 5e-2 $\rightarrow$ 1e-4 improves accuracy at the cost of iters.
EQ samples n_{eq}	{4..64}	$\uparrow$	H3D $N=64$: $n_{\text{eq}} = 32 \rightarrow 64$ doubles coverage at no accuracy cost.

Table 4: Knobs that push *speedup*. Direction is the way to turn the knob to increase wall-clock speedup. The first three rows are deployment-time knobs (no retraining); the last three are engineering optimizations.

Knob	Range	Direction	Evidence
<i>Deployment-time (no retraining):</i>			
GN max iters	{3..50}	$\downarrow$	P3D $N=64$: max_iters 20 $\rightarrow$ 11 takes speedup 51 $\rightarrow$ 96 $\times$ at same accuracy.
GN tolerance	{ 10^{-2} .. 10^{-1} }	$\uparrow$ (looser)	P3D $N=128$ Run3: tol 1e-4 $\rightarrow$ 5e-2 takes speedup 57 $\rightarrow$ 109 $\times$.
EQ samples n_{eq}	{4..64}	$\downarrow$	H3D $N=32$ Run12: $n_{\text{eq}} = 4$ extends speedup frontier.

in Table 3 push the achievable frontier downward but require a fresh training run per setting. The engineering row of Table 4 contributes the bulk of our best-speedup numbers on Heat-3D — in particular, the κ -as-runtime-arg JAX-tracing fix on its own takes the trained Heat-3D $N = 128$ model from $14.7\times$ to $191\times$ wall-clock, with bit-identical accuracy.

Limitations. (i) All cells are evaluated at a single random seed; we do not report variance across runs. (ii) Discretisations are finite-difference on uniform Cartesian grids; transfer to unstructured FEM meshes and curved geometries is untested and we expect the CP spatial head to require modification. (iii) Benchmarks are restricted to the elliptic Poisson and parabolic heat equations; advection-dominated problems (Burgers, shallow water, wave) are not evaluated and would exercise solver-side machinery our framework does not currently include. (iv) Baseline hyperparameters are taken from canonical configurations rather than independently re-swept per cell. (v) Heat rollouts are 50 steps; long-horizon stability beyond this window is not assessed. (vi) Training cost ranges from ~ 10 minutes (Poisson-3D $N=64$) to ~ 8 hours (Heat-2D $N=128$, 300,000 epochs) on a single A100; the amortisation argument is implicit in many-query deployment.

7 Conclusion and Future Work

We built an NM-ROM for elliptic and parabolic PDEs whose distinguishing property is a deployment-time accuracy/speed Pareto curve traced by a single trained model: at inference, three solver-side knobs — the Gauss-Newton iteration cap, the Gauss-Newton early-exit tolerance, and the Empirical Quadrature sample count — continuously trade accuracy for speedup. This is the lever neural operators do not expose: each trained FNO or DeepONet delivers one fixed operating point. The framework is faster than full-order conjugate gradient and beats both neural-operator baselines on accuracy and speed at every measured cell except one. Mechanically, the decoder is a learned non-linear manifold that lifts the Kolmogorov n -width barrier of linear POD; the PDE itself is enforced by Galerkin projection of the discrete residual onto that manifold; Dirichlet boundary conditions are enforced exactly by a binary mask in the decoder; Empirical Quadrature decouples residual cost from mesh size; and the whole pipeline is matrix-free in JAX.

Three pieces had to come together to make this work. First, an architectural choice — the LinearCPDecoder (linear skip + shallow MLP + rank- R canonical-polyadic head) feeding from a ViT encoder — that simultaneously satisfies the cold-start Gauss-Newton convergence, EQ-compatibility, and sub-cubic parameter-scaling constraints that prior NM-ROM decoders could not satisfy together at 3D resolution. Second, exact Dirichlet enforcement by decoder masking, which lets the Galerkin projection annihilate boundary equations as in classical projection methods rather than via soft penalties. Third, a JAX implementation that evaluates every projected operator matrix-free through forward- and reverse-mode automatic differentiation, removing the dense-Jacobian bottleneck that historically pinned NM-ROMs to small problems.

Across the elliptic Poisson and parabolic heat equations in 2D and 3D the framework delivers wall-clock speedups over full-order conjugate gradient from $36\times$ to $324\times$ at engineering-grade accuracy. A natural next step is to evaluate the same framework on advection-dominated benchmarks — 1D and 2D Burgers, shallow water — where the n -width gap between linear and non-linear manifolds is largest, and where existing neural operators also tend to be unreliable. Extending the spatial head beyond Cartesian tensor products to support unstructured meshes is a second open direction.

References

- [1] J. Barnett and C. Farhat. Quadratic approximation manifold for mitigating the Kolmogorov barrier in nonlinear projection-based model order reduction. *Journal of Computational Physics*, 464:111348, 2022.
- [2] P. Benner, S. Gugercin, and K. Willcox. A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Review*, 57(4):483–531, 2015.
- [3] G. Berkooz, P. Holmes, and J. L. Lumley. The proper orthogonal decomposition in the analysis of turbulent flows. *Annual Review of Fluid Mechanics*, 25(1):539–575, 1993.

- [4] S. Berrone, C. Canuto, M. Pintore, and N. Sukumar. Enforcing Dirichlet boundary conditions in physics-informed neural networks and variational physics-informed neural networks. *arXiv preprint arXiv:2210.14795*, 2022.
- [5] N. Boullé and A. Townsend. A mathematical guide to operator learning. In *Handbook of Numerical Analysis*, volume 25, pages 83–125. Elsevier, 2024.
- [6] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [7] R. Brecht, R. O. Popovych, A. Bihlo, and Y. Popovych. Improving physics-informed DeepONets with hard constraints. *arXiv preprint arXiv:2309.07899*, 2023.
- [8] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han. Once-for-All: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations*, 2020.
- [9] S. Cao. Choose a Transformer: Fourier or Galerkin. In *Advances in Neural Information Processing Systems*, 2021.
- [10] K. Carlberg, C. Bou-Mosleh, and C. Farhat. Efficient non-linear model reduction via a least-squares Petrov–Galerkin projection and compressive tensor approximations. *International Journal for Numerical Methods in Engineering*, 86(2):155–181, 2011.
- [11] S. Chaturantabut and D. C. Sorensen. Nonlinear model reduction via discrete empirical interpolation. *SIAM Journal on Scientific Computing*, 32(5):2737–2764, 2010.
- [12] W. Chen, J. Song, P. Ren, S. Subramanian, D. Morozov, and M. W. Mahoney. Data-efficient operator learning via unsupervised pretraining and in-context learning. In *Advances in Neural Information Processing Systems*, 2024.
- [13] A. Cohen and R. DeVore. Approximation of high-dimensional parametric PDEs. *Acta Numerica*, 24:1–159, 2015.
- [14] A. N. Diaz, Y. Choi, and M. Heinkenschloss. A fast and accurate domain-decomposition nonlinear manifold reduced order model. *Computer Methods in Applied Mechanics and Engineering*, 425:116943, 2024.
- [15] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- [16] M. S. Eshaghi, C. Anitescu, N. Valizadeh, Y. Wang, X. Zhuang, and T. Rabczuk. Nows: Neural operator warm starts for accelerating iterative solvers. *arXiv preprint arXiv:2511.02481*, 2025.
- [17] C. Farhat, P. Avery, T. Chapman, and J. Cortial. Dimensional reduction of nonlinear finite element dynamic models with finite rotations and energy-based mesh sampling and weighting for computational efficiency. *International Journal for Numerical Methods in Engineering*, 98(9):625–662, 2014.
- [18] S. Fresca and A. Manzoni. POD-DL-ROM: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388:114181, 2022.
- [19] R. Geelen, S. Wright, and K. Willcox. Operator inference for non-intrusive model reduction with quadratic manifolds. *Computer Methods in Applied Mechanics and Engineering*, 403:115717, 2022.
- [20] C. Greif and K. Urban. Decay of the Kolmogorov N -width for wave problems. *Applied Mathematics Letters*, 96:216–222, 2019.
- [21] A. Griewank and A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, Philadelphia, 2nd edition, 2008.

- [22] Z. Hao, Z. Wang, H. Su, C. Ying, Y. Dong, S. Liu, Z. Cheng, J. Song, and J. Zhu. GNOT: A general neural operator transformer for operator learning. In *International Conference on Machine Learning*, 2023.
- [23] Z. Hao, C. Su, S. Liu, J. Berner, C. Ying, H. Su, A. Anandkumar, J. Song, and J. Zhu. DPOT: Auto-regressive denoising operator transformer for large-scale PDE pre-training. In *International Conference on Machine Learning*, 2024.
- [24] M. Herde, B. Raonic, T. Rohner, R. Käppeli, R. Molinaro, E. de Bezenac, and S. Mishra. Poseidon: Efficient foundation models for PDEs. In *Advances in Neural Information Processing Systems*, 2024.
- [25] J. A. Hernández, M. A. Caicedo, and A. Ferrer. Dimensional hyper-reduction of nonlinear finite element models via empirical cubature. *Computer Methods in Applied Mechanics and Engineering*, 313:687–722, 2017.
- [26] M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952.
- [27] J. S. Hesthaven, G. Rozza, and B. Stamm. *Certified Reduced Basis Methods for Parametrized Partial Differential Equations*. SpringerBriefs in Mathematics. Springer, 2016.
- [28] M. Hirsch, F. Pichi, and J. S. Hesthaven. Neural empirical interpolation method for nonlinear model reduction. *arXiv preprint arXiv:2406.03562*, 2024.
- [29] B. Holzsuh, Q. Liu, G. Kohl, and N. Thuerey. PDE-Transformer: Efficient and versatile transformers for physics simulations. In *International Conference on Machine Learning*, 2025.
- [30] W. Hu, S. Liu, P. Qiao, Z. Sun, and Y. Dou. Transolver is a linear transformer: Revisiting physics-attention through the lens of linear attention. *arXiv preprint arXiv:2511.06294*, 2025.
- [31] T. J. R. Hughes. *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover Publications, 2000.
- [32] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3:422–440, 2021.
- [33] Y. Kim, Y. Choi, D. Widemann, and T. Zohdi. A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder. *Journal of Computational Physics*, 451:110841, 2022.
- [34] M. Kim, T. Wen, K. Lee, and Y. Choi. Physics-informed reduced order model with conditional neural fields. *arXiv preprint arXiv:2412.05233*, 2024.
- [35] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, and S. Hoyer. Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21):e2101784118, 2021.
- [36] T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- [37] A. Kopaničáková and G. E. Karniadakis. DeepOnet based preconditioning strategies for solving parametric linear systems of equations. *arXiv preprint arXiv:2401.02016*, 2024.
- [38] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: learning maps between function spaces. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- [39] C. L. Lawson and R. J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, Englewood Cliffs, NJ, 1974.
- [40] K. Lee and K. T. Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.

- [41] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- [42] Z. Li, D. Z. Huang, B. Liu, and A. Anandkumar. Fourier neural operator with learned deformations for PDEs on general geometries. *Journal of Machine Learning Research*, 24(388):1–26, 2023.
- [43] Z. Li, K. Meidani, and A. B. Farimani. Transformer for partial differential equations’ operator learning. *Transactions on Machine Learning Research*, 2023.
- [44] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, and A. Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM/IMS Journal of Data Science*, 1(3):1–27, 2024.
- [45] P. Lippe, B. Veeling, P. Perdikaris, R. E. Turner, and J. Brandstetter. PDE-Refiner: Achieving accurate long rollouts with neural PDE solvers. In *Advances in Neural Information Processing Systems*, 2023.
- [46] I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.
- [47] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [48] W. Luo, Y. Li, R. Urtasun, and R. Zemel. Understanding the effective receptive field in deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, 2016.
- [49] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- [50] M. McCabe, B. Régald-Saint Blancard, L. H. Parker, R. Ohana, M. Cranmer, A. Bietti, M. Eickenberg, S. Golkar, G. Krawezik, F. Lanusse, M. Pettee, T. Tesileanu, K. Cho, and S. Ho. Multiple physics pretraining for physical surrogate models. In *Advances in Neural Information Processing Systems*, 2024.
- [51] M. A. Mirhoseini and M. J. Zahr. Accelerated solutions of convection-dominated partial differential equations using implicit feature tracking and empirical quadrature. *arXiv preprint arXiv:2305.15661*, 2023.
- [52] F. Negri, A. Manzoni, and D. Amsallem. Efficient model reduction of parametrized systems by matrix discrete empirical interpolation. *Journal of Computational Physics*, 303:431–454, 2015.
- [53] N. C. Nguyen. High-order empirical interpolation methods for real time solution of parametrized nonlinear PDEs. *arXiv preprint arXiv:2410.02100*, 2024.
- [54] J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, New York, 2nd edition, 2006.
- [55] A. Novikov, D. Podoprikin, A. Osokin, and D. P. Vetrov. Tensorizing neural networks. In *Advances in Neural Information Processing Systems*, 2015.
- [56] M. Ohlberger and S. Rave. Reduced basis methods: success, limitations and future challenges. In *Proceedings of the ALGORITHMY Conference*, 2016.
- [57] A. Quarteroni, A. Manzoni, and F. Negri. *Reduced Basis Methods for Partial Differential Equations: An Introduction*. UNITEXT, Vol. 92. Springer, 2015.
- [58] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks. *Journal of Computational Physics*, 378:686–707, 2019.
- [59] F. Romor, G. Stabile, and G. Rozza. Non-linear manifold reduced-order models with convolutional autoencoders and reduced over-collocation method. *Journal of Scientific Computing*, 94:74, 2023.

- [60] Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2nd edition, 2003.
- [61] L. J. Shikhman. One operator to rule them all? On boundary-indexed operator families in neural PDE solvers. *arXiv preprint arXiv:2603.01406*, 2026.
- [62] L. Sirovich. Turbulence and the dynamics of coherent structures, Parts I–III. *Quarterly of Applied Mathematics*, 45(3):561–590, 1987.
- [63] N. Sukumar and A. Srivastava. Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *Computer Methods in Applied Mechanics and Engineering*, 389:114333, 2022.
- [64] A. Tran, A. Mathews, L. Xie, and C. S. Ong. Factorized Fourier neural operators. In *International Conference on Learning Representations*, 2023.
- [65] K. Um, R. Brand, Y. Fei, P. Holl, and N. Thuerey. Solver-in-the-loop: Learning from differentiable physics to interact with iterative PDE-solvers. In *Advances in Neural Information Processing Systems*, 2020.
- [66] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [67] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.
- [68] S. Wen, A. Kumbhat, L. Lingsch, S. Mousavi, Y. Zhao, P. Chandrashekar, and S. Mishra. Geometry aware operator transformer as an efficient and accurate neural surrogate for PDEs on arbitrary domains. In *Advances in Neural Information Processing Systems*, 2025.
- [69] T. Wang and C. Wang. Latent neural operator for solving forward and inverse PDE problems. In *Advances in Neural Information Processing Systems*, 2024.
- [70] H. Wang, J. Li, A. Dwivedi, K. Hara, and T. Wu. BENO: Boundary-embedded neural operators for elliptic PDEs. In *International Conference on Learning Representations*, 2024.
- [71] H. Wu, H. Luo, H. Wang, J. Wang, and M. Long. Transolver: A fast transformer solver for PDEs on general geometries. In *International Conference on Machine Learning*, 2024.
- [72] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu. On layer normalization in the Transformer architecture. In *International Conference on Machine Learning*, 2020.
- [73] M. Yano and A. T. Patera. An LP empirical quadrature procedure for reduced basis treatment of parametrized nonlinear PDEs. *Computer Methods in Applied Mechanics and Engineering*, 344:1104–1123, 2019.
- [74] J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang. Slimmable neural networks. In *International Conference on Learning Representations*, 2019.

A Per-cell Training Schedule

This appendix lists the per-cell training schedule referenced in §4. The architectural hyperparameters and the optimiser family (AdamW [47] with a linear warm-up to a peak rate, cosine decay [46] to 10^{-6}) are fixed across every cell; what varies is the encoder/decoder sizing, the epoch budget, the batch size, and a few cell-specific deviations noted below. Warm-up is always $\min(500, \lfloor \text{epochs}/10 \rfloor)$ steps. After training, EQ weights are computed from training snapshots via `scipy.optimize.nnls` with a minimum support size of 300 nodes: 80 snapshots for Poisson and 32 snapshot rows for Heat.

Poisson-2D ($N \in \{64, 128, 256\}$). Default schedule: 100,000 epochs, batch 32, peak learning rate 10^{-3} , weight decay 5×10^{-4} . The same trained checkpoint serves both the best-accuracy and best-speedup operating points in Table 2; the two points differ only in the inference-time GN iteration cap and tolerance.

Poisson-3D, $N \in \{32, 64\}$. Default schedule (as Poisson-2D). At $N=64$ the EQ NNLS system is assembled from $n_{\text{snap}} = 500$ –600 training snapshots rather than the default 80; in our sweeps this lowered the accuracy-axis floor without changing speedup.

Poisson-3D, $N=128$. The two operating points in Table 2 use distinct training recipes from the same architecture family. The *best-accuracy* point ($k=16$, $R=512$) uses 60,000 epochs of fresh training (seed 42, lr 10^{-3} , wd 5×10^{-4}) followed by 30,000–60,000 epochs of fine-tuning at lr 5×10^{-5} , wd 2×10^{-3} ; this two-stage schedule was the only one in our sweep that pushed the held-out rel- L^2 below the fresh-train floor. The *best-speedup* point ($k=16$, $R=768$) uses fresh-only training for 60,000 epochs at the default rate; here the binding constraint is the CP rank, not the training horizon, and fine-tuning gave no further gain. We also observed that this resolution is sensitive to the random seed: alternative seeds at the same recipe produced rel- L^2 several times worse on at least one held-out test case. The published runs use seed 42.

Heat-2D ($N \in \{64, 128, 256\}$). Common across all three resolutions: $k=64$, CP rank $R=256$, peak lr 10^{-3} , wd 2×10^{-3} ; the encoder/decoder sizing matches the architecture in §3.4 except for an enlarged ViT embedding dimension (96). The epoch budget and batch size scale with resolution: 80,000 epochs at batch 32 for $N=64$, 150,000 at batch 16 for $N=128$, and 100,000 at batch 8 for $N=256$ (batch reduction is memory-driven). At $N=128$ we verified that 300,000 epochs overtrains the manifold; 150,000 is the sweet spot. As in Poisson-2D, the best-accuracy and best-speedup points within each cell are produced from the same checkpoint, with only inference-time GN settings differing.

Heat-3D, $N \in \{32, 64\}$. Single-stage training at 80,000 epochs ($N=32$, $k=32$, $R=512$) and 100,000 epochs ($N=64$, $k=40$, $R=512$), both at default rates. Speed-axis variation is again obtained at inference by changing the GN iteration cap and tolerance from a single trained checkpoint.

Heat-3D, $N=128$. Two-stage training: 60,000 epochs of fresh training ($k=40$, $R=512$, default rates), followed by 40,000 epochs of fine-tuning at lr 5×10^{-4} . Empirically, the rollout loss surface conditions improve substantially after the fresh stage stabilises the manifold, which is why the second stage helps. We note that the val rel- L^2 floor at this cell appears to be set by manifold reconstruction error on low- κ test cases rather than by the training horizon: an extensive sweep over rank, encoder size, fine-tuning length, and κ -weighted training did not move it substantially beyond the value reported in Table 2.

B Architecture Choices: Evidence from Sweeps

This appendix collects the empirical evidence that motivates the three architectural choices in §3.4: the linear skip in the decoder, the ViT encoder, and the CP-tensor head.

Cold-start GN failure of pure-MLP decoders. We trained the shallow masked autoencoder NM-ROM (SMA-NM-ROM) of Kim et al. [33] on Poisson-2D $N=64$ snapshots using the published recipe (plain Adam, lr 10^{-3} , batch size 240). The autoencoder reconstructs unseen validation snapshots at 1.4% relative- L^2 error, confirming the manifold has sufficient capacity. Latent Gauss-Newton from $z = 0$ nevertheless diverges to rel- $L^2 = 1.4$, while warm-starting from $z = \mathcal{E}(u_{\text{FOM}})$ (an oracle that already uses the FOM solution we are trying to predict) converges to 1.8×10^{-2} in three iterations. Levenberg-Marquardt damping at $\lambda = 10^{-3}$ does not restore cold-start convergence. The linear skip in our LinearCPDecoder eliminates this failure mode at every measured cell.

POD baseline on Poisson-3D. The MLP branch in our decoder is what gives the manifold its non-linearity; without it the decoder reduces to POD. At $k = 16$ on Poisson-3D, POD gives 39% relative error, against 0.6% for the full LinearCPDecoder at the same latent dimension. We keep the

MLP shallow (two layers) on purpose: deeper $z \rightarrow h$ maps are more expressive but make $J_{\mathcal{D}}$ vary more across the latent space, which hurts cold-start GN.

CNN vs. ViT encoder, matched parameter budget. On 700 Poisson-3D samples a convolutional encoder plateaus at 6–10% validation $\text{rel-}L^2$ regardless of regularisation; a matched-parameter-budget ViT reaches sub-percent error. The mechanism is straightforward: a CNN sees only a local patch per layer and needs many layers to reach across the grid, whereas each ViT attention layer is global by construction.

Architecture-sweep failure modes. Two natural decoder variants we tried during architecture sweeps fail at $N_g = 128$ in 3D. (i) An MLP-only-into-CP decoder (no linear skip) gives the cold-start GN divergences described above on every measured cell, even when the autoencoder reconstruction error is well below 1%. (ii) A linear-skip-into-dense-readout decoder runs out of memory before training finishes: the dense readout to a 128^3 grid costs $\sim 10^9$ parameters, against $\sim 200\text{K}$ for the CP head at the same rank.

SMA-NM-ROM scaling cliff. The same scaling argument is visible empirically in the SMA-NM-ROM of Kim et al. [33], whose fully dense $\text{Linear}(m, M_1)$ encoder with $M_1 = 2m$ requires ~ 65 GB even after capping M_1 at 8192 at $N = 128$ in 3D, exceeding the memory of a single A100 GPU; the published $M_1 = 2m$ is unbuildable on any current accelerator. Our ViT encoder (sub-cubic parameters in mesh side) and CP-tensor readout (linear in mesh side) avoid the memory cliff: at $N_g = 128$ in 3D, total VRAM for our pipeline is < 30 GB.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction’s three numbered contributions (deployment-time accuracy/speed knobs, the matrix-free Galerkin + EQ + masked-decoder NM-ROM framework, and the LinearCPDecoder architecture) are each backed by a corresponding results subsection: Pareto dominance over FNO/DeepONet/SMA-NM-ROM in Table 1 (§6.1), the architecture choices in §3.4, and the deployment-time tunability in §6.3. Claims are scoped to elliptic and parabolic PDEs (Poisson and Heat in 2D and 3D); we do not claim coverage of advection-dominated regimes.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: §6.3 enumerates six concrete limitations (single-seed evaluation, finite-difference uniform-grid restriction, scope limited to elliptic/parabolic PDEs, baseline hyperparameter choices, 50-step Heat rollout horizon, and per-cell training cost), and §7 explicitly scopes the framework to elliptic and parabolic PDEs and flags advection-dominated benchmarks (Burgers, shallow water) as outside the tested regime. Quantitative limitations (e.g., the Heat-3D $N=128$ rel- $L^2 \sim 1.8 \times 10^{-1}$ floor and the higher-frequency Poisson-3D outliers) are reported directly in Table 1 rather than hidden.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper presents an architectural and algorithmic contribution validated empirically; it does not state new theorems. The classical Galerkin projection, Gauss–Newton iteration, and EQ/NNLS hyper-reduction it composes are referenced to their original sources (§3, §3.3).

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.

- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: §5 specifies the PDE problems, parameter distributions, train/validation splits, FOM solver tolerances, and ROM solver settings; §4 gives the optimizer, learning-rate schedule, encoder/decoder hyperparameters, batch size, epoch counts, and EQ NNLS protocol; Table 2 lists the per-cell (k, R, N_{eq}) values that vary across cells. Per-cell deployment knobs (GN iterations, GN tolerance, EQ sample count) are described in §6.3.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Code and trained checkpoints are not released with the submission to preserve double-blind review. Datasets are synthetic (§5 fully specifies the analytic Poisson forcings and Heat initial-condition distributions); together with the architectural and training details in §4, the experiments are reproducible from the paper alone. We plan to release the JAX implementation and snapshots upon de-anonymization.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: §4 specifies AdamW with weight decay 5×10^{-4} , a warmup-cosine schedule peaking at 10^{-3} , patch size 8^d , embedding dim 64, $L = 4$ transformer blocks, 4 attention heads, and decoder hidden width 256; §5 specifies 500/100 train/val split for Poisson and 500/50 trajectories for Heat (with 100/10 at $N=128$ for Heat memory), and the GN/CG tolerances used at test time; Table 2 lists per-cell (k, R, N_{eq}) .

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Reported relative- L^2 errors are means (or medians, in the Poisson-3D case) over the held-out validation set described in §5; we do not report formal error bars or confidence intervals because each (PDE, N) cell already aggregates over many random forcings or initial conditions and the measured cell-to-cell variation is much larger than within-cell variation. For Poisson-3D we explicitly use 10-case medians (LHS, seed=9999) over $(k_1, k_2, k_3) \sim \mathcal{U}[1, 3]^3$ to avoid sensitivity to high-frequency outliers; the protocol is documented in the Table 1 caption.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: All FOM, NM-ROM, and neural-operator runs use a single NVIDIA A100 GPU (§5, §6). Training cost is given in §4 (e.g., ~ 10 min for Poisson-3D $N = 64$ at 100k epochs; 40k epochs for Heat-3D); inference wall-clock is reported per cell in Table 1. §3.4 also reports the peak VRAM gap that motivated our architecture choices (< 30 GB for our pipeline at $N=128$ in 3D vs. ~ 65 GB for the SMA-NM-ROM baseline).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work develops a numerical method for elliptic and parabolic PDEs benchmarked on synthetic data; it does not involve human subjects, scraped data, or sensitive applications, and we have reviewed the NeurIPS Code of Ethics for full conformance.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This is foundational scientific-computing methodology for diffusion-type PDEs; positive impacts are diffuse (faster engineering simulation, reduced compute for ROM-based design loops) and we are not aware of a direct path to negative societal applications. The method does not generate content, identify individuals, or replace human judgement.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artefacts (a JAX implementation and trained PDE-surrogate decoders) are PDE solvers with no high-risk misuse profile; no scraped data, generative content, or pretrained foundation models are released.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All third-party software and methods are credited to their original publications: JAX [6], FNO [41], DeepONet [49], and the SMA-NM-ROM recipe of Kim et al. [33]. We rely on the standard permissive licenses (Apache-2.0/MIT) under which these projects are distributed; no datasets are scraped or re-packaged.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: No new assets (datasets, pretrained models, or benchmarks) are released with this submission. If accepted, we plan to release the JAX implementation and a small set of trained decoder checkpoints with documentation upon de-anonymization; these are not part of the submitted artefacts.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human-subjects research; all data are synthetic snapshots from finite-difference PDE solves.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects are involved, so IRB review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLM-based coding assistants were used during experimental development to help draft and refactor implementation code (training scripts, data-generation pipelines, plotting and benchmarking utilities) and during early-stage ideation to brainstorm decoder-architecture variants and ablation candidates. All resulting code was reviewed, debugged, and validated by the authors against the numerical results reported in the paper, and all design decisions retained in the final architecture (linear-skip + shallow-MLP + canonical-polyadic decoder, masked Dirichlet enforcement, EQ hyper-reduction, JAX matrix-free Galerkin projection) were selected and verified by the authors based on the empirical sweeps reported in §6. LLMs were not used as components of the deployed models, the FOM/ROM solvers, or the experimental analysis itself.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.